

Parametric Curves & Bézier Curves

From Lines to Smooth Curves in Computer Graphics

Ashrafur Rahman

Adjunct Lecturer

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

Introduction

Parametric Lines

Higher Order Polynomials

Curve Continuity

Bézier Curves

General Bézier Curves

Bernstein Polynomials

Matrix Representation

Applications

Properties & Limitations

Introduction

Why Parametric Curves?

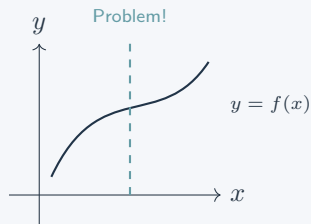
Problems with Traditional Representations

Explicit form: $y = f(x)$

- Single-valued functions only
- Cannot represent vertical lines
- Limited to function graphs
- No natural parameterization

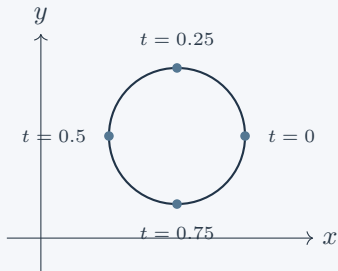
Implicit form: $F(x, y) = 0$

- Difficult to sample points
- Hard to control shape
- No direction information
- Complex for animations



Cannot represent vertical lines

Parametric Representation: The Solution



Parametric Form

$\mathbf{P}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$ Where $t \in [0, 1]$ is the parameter

Advantages of Parametric Curves

- Can represent any curve shape (circles, loops, vertical lines)
- Natural parameterization for animations

Applications in Computer Graphics

2D Graphics

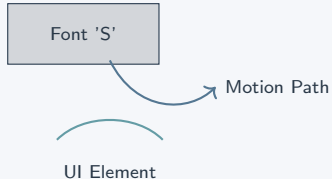
- **Font Rendering:** TrueType, OpenType fonts
- **Vector Graphics:** SVG, Adobe Illustrator
- **UI Design:** Smooth transitions, icons
- **Logo Design:** Scalable curves

Animation & Motion

- **Motion Paths:** Object trajectories
- **Camera Movement:** Smooth camera paths
- **Interpolation:** Between keyframes
- **Easing Functions:** CSS animations

3D Modeling

- **Surface Generation:** NURBS surfaces
- **CAD Systems:** Engineering design
- **Game Development:** Character modeling
- **Architecture:** Curved structures



Parametric Lines

Parametric Line Representation

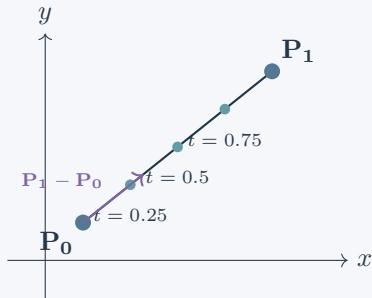
Linear Parametric Equation

Given two points $\mathbf{P}_0$ and $\mathbf{P}_1$:

$$\begin{aligned}\mathbf{P}(t) &= \mathbf{P}_0 + t(\mathbf{P}_1 - \mathbf{P}_0) \\ &= (1 - t)\mathbf{P}_0 + t\mathbf{P}_1\end{aligned}$$

Where $t \in [0, 1]$:

- $t = 0 \Rightarrow \mathbf{P}(0) = \mathbf{P}_0$
- $t = 1 \Rightarrow \mathbf{P}(1) = \mathbf{P}_1$
- $t = 0.5 \Rightarrow$ midpoint



Linear Interpolation (LERP)

LERP Function

Linear interpolation between two values:

$$\text{lerp}(A, B, t) = (1 - t)A + tB$$

Properties:

- $\text{lerp}(A, B, 0) = A$
- $\text{lerp}(A, B, 1) = B$
- Linear relationship
- Convex combination

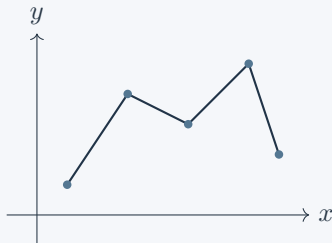
Code Example

```
def lerp(A, B, t):  
    return (1 - t) * A + t * B
```

Example usage

Interactive Demo

[[Desmos Demo: Piecewise Lines](#)]



Piecewise Linear Curve

Higher Order Polynomials

From Linear to Polynomial

Polynomial Curves

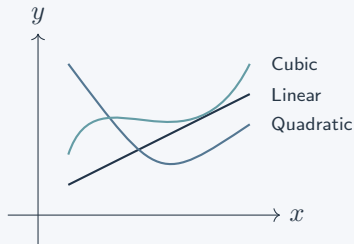
Linear: $P(t) = a_0 + a_1t$

Quadratic: $P(t) = a_0 + a_1t + a_2t^2$

Cubic: $P(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

Benefits:

- More control over shape
- Smoother curves
- Higher flexibility



[Desmos Demo: Piecewise Polynomials]

Demonstrate how polynomial pieces can create complex shapes

The Control Problem

Challenges with Polynomial Coefficients

Unintuitive Control:

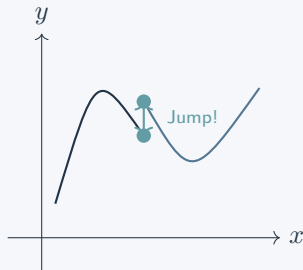
- Coefficients a_0, a_1, a_2, a_3 don't correspond to visual features
- Hard to predict curve shape from coefficients
- Difficult to achieve desired curves

Global Effects:

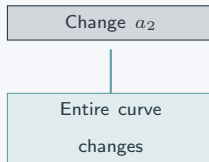
- Changing one coefficient affects entire curve
- No local control
- Unpredictable modifications

Piecewise Issues:

- Discontinuities at joints
- Visible "kinks" or jumps
- Not visually pleasing



Discontinuous Connection



Curve Continuity

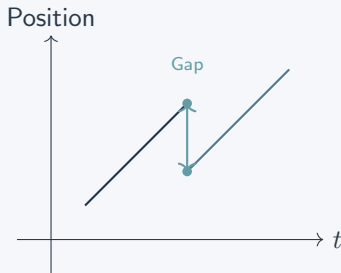
What is Continuity?

Mathematical Definition

Two curve segments $\mathbf{P}_1(t)$ and $\mathbf{P}_2(t)$ are continuous at their junction if they satisfy certain conditions.

Levels of Continuity:

- C^0 : Position continuity
- C^1 : Tangent continuity
- C^2 : Curvature continuity
- C^n : n -th derivative continuity



Not C^0 continuous

C^0 Continuity: Position

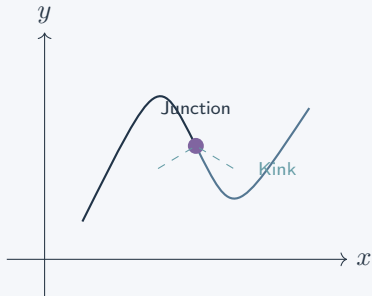
C^0 Continuity

The curves meet at the same point:

$$\mathbf{P}_1(1) = \mathbf{P}_2(0)$$

Properties:

- No gaps in the curve
- Connected but may have kinks
- Minimum requirement for smooth appearance



C^0 Continuous

C^1 Continuity: Tangent

C^1 Continuity

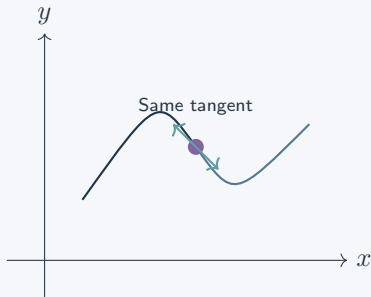
Same position AND same tangent:

$$\mathbf{P}_1(1) = \mathbf{P}_2(0)$$

$$\mathbf{P}_1'(1) = \mathbf{P}_2'(0)$$

Properties:

- No kinks or sharp corners
- Smooth tangent transition
- Visually pleasing curves
- Same velocity at junction



C^1 Continuous

C^2 Continuity: Curvature

C^2 Continuity

Same position, tangent, AND curvature:

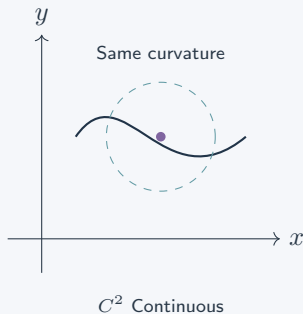
$$\mathbf{P}_1(1) = \mathbf{P}_2(0)$$

$$\mathbf{P}_1'(1) = \mathbf{P}_2'(0)$$

$$\mathbf{P}_1''(1) = \mathbf{P}_2''(0)$$

Properties:

- No sudden change in bending
- Smoothest visual appearance
- Required for high-quality surfaces
- Same acceleration at junction



Why Continuity Matters

Visual Quality: Higher continuity = smoother, more natural appearance

Bézier Curves

Quadratic Bézier: LERP of LERPs

Construction Method

Given three control points P_0 , P_1 , P_2 :

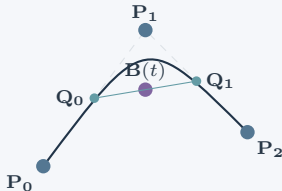
Step 1: Linear interpolation

$$Q_0(t) = \text{lerp}(P_0, P_1, t)$$

$$Q_1(t) = \text{lerp}(P_1, P_2, t)$$

Step 2: Another linear interpolation

$$B(t) = \text{lerp}(Q_0, Q_1, t)$$



[[Desmos Demo: Quadratic Bézier Construction](#)]

Quadratic Bézier: Mathematical Form

Expanded Form

Starting from the LERP of LERPs:

$$\begin{aligned}\mathbf{B}(t) &= (1-t)[(1-t)\mathbf{P}_0 + t\mathbf{P}_1] + t[(1-t)\mathbf{P}_1 + t\mathbf{P}_2] \\ &= (1-t)^2\mathbf{P}_0 + 2t(1-t)\mathbf{P}_1 + t^2\mathbf{P}_2\end{aligned}$$

Bernstein Form:

$$\mathbf{B}(t) = B_{0,2}(t)\mathbf{P}_0 + B_{1,2}(t)\mathbf{P}_1 + B_{2,2}(t)\mathbf{P}_2$$

Where the Bernstein polynomials are:

$$B_{0,2}(t) = (1-t)^2 \quad B_{1,2}(t) = 2t(1-t) \quad B_{2,2}(t) = t^2$$

Key Properties

i

- **Endpoint Interpolation:** $\mathbf{B}(0) = \mathbf{P}_0$, $\mathbf{B}(1) = \mathbf{P}_2$

Cubic Bézier Curves

Cubic Bézier Formula

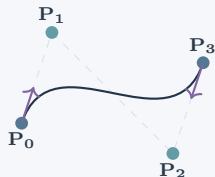
Four control points

P_0, P_1, P_2, P_3 :

$$\begin{aligned} B(t) = & (1-t)^3 P_0 \\ & + 3t(1-t)^2 P_1 \\ & + 3t^2(1-t) P_2 \\ & + t^3 P_3 \end{aligned}$$

Control Point Roles:

- P_0 : Start point
- P_1 : Start tangent control
- P_2 : End tangent control
- P_3 : End point



Most common in graphics

General Bézier Curves

General Bézier Definition

General Formula

A Bézier curve of degree n with control points $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$:

$$\mathbf{B}(t) = \sum_{i=0}^n B_{i,n}(t) \mathbf{P}_i$$

Where $B_{i,n}(t)$ are the Bernstein basis polynomials:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

And $\binom{n}{i} = \frac{n!}{i!(n-i)!}$ is the binomial coefficient.

Degree vs Number of Control Points

- **Linear:** 2 control points, degree 1
- **Quadratic:** 3 control points, degree 2
- **Cubic:** 4 control points, degree 3

Fundamental Properties

Bézier Properties

1. Endpoint Interpolation:

- $B(0) = P_0$
- $B(1) = P_n$

2. Convex Hull Property:

- Curve lies within convex hull of control points
- Provides predictable bounds

3. Variation Diminishing:

- Curve doesn't oscillate more than control polygon
- Smooth, well-behaved curves

4. Affine Invariance:

- Transform control points = transform curve
- Works with any coordinate system

Convex Hull



Degree 4 Bézier



Why Bézier Curves Work So Well

Intuitive Control

Visual Predictability:

- Control points have geometric meaning
- Pulling a control point "attracts" the curve
- Natural, intuitive manipulation

Tangent Control:

- First and last segments of control polygon define tangents
- $B'(0) \propto (P_1 - P_0)$
- $B'(1) \propto (P_n - P_{n-1})$

Stability:

- Robust to numerical errors
- Well-conditioned algorithms
- Consistent behavior



Bernstein Polynomials

Bernstein Basis Functions

Bernstein Polynomials

The i -th Bernstein polynomial of degree n :

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

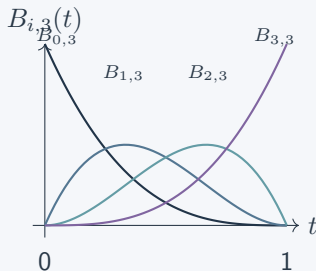
Examples for degree 3:

$$B_{0,3}(t) = (1-t)^3$$

$$B_{1,3}(t) = 3t(1-t)^2$$

$$B_{2,3}(t) = 3t^2(1-t)$$

$$B_{3,3}(t) = t^3$$



Remarkable Properties

Key Properties

1. Non-negative:

$$B_{i,n}(t) \geq 0 \text{ for } t \in [0, 1]$$

2. Partition of Unity:

$$\sum_{i=0}^n B_{i,n}(t) = 1$$

3. Endpoint Values:

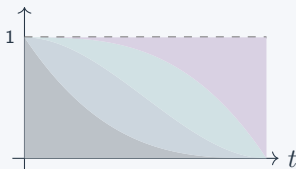
$$B_{i,n}(0) = \begin{cases} 1 & \text{if } i = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$B_{i,n}(1) = \begin{cases} 1 & \text{if } i = n \\ 0 & \text{otherwise} \end{cases}$$

4. Maximum at:

$$t = \frac{i}{n}$$

Partition of Unity



Sum always equals 1



Always ≥ 0

Why These Properties Matter

Geometric Interpretation

! **Convex Combination:** Since Bernstein polynomials are non-negative and sum to 1, any point on a Bézier curve is a *convex combination* of the control points.

$$\mathbf{B}(t) = \sum_{i=0}^n \underbrace{B_{i,n}(t)}_{\text{weights}} \mathbf{P}_i$$

This means the curve point is a "weighted average" of control points, where weights are always positive and sum to 1.

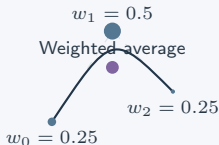
Consequences

1. Convex Hull Property:

- Curve never "escapes" control polygon
- Provides natural bounds

2. Stability:

- Small changes in control points
→ small changes in curve



Matrix Representation

Matrix Form for Efficiency

Cubic Bézier Matrix Form

For a cubic Bézier curve:

$$\mathbf{B}(t) = \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}_2 \\ \mathbf{P}_3 \end{bmatrix}$$

Or more compactly:

$$\mathbf{B}(t) = \mathbf{T}(t) \cdot \mathbf{M}_B \cdot \mathbf{P}$$

Matrix Components

$\mathbf{T}(t)$: Parameter vector

$$\mathbf{T}(t) = [1 \quad t \quad t^2 \quad t^3]$$

$\mathbf{M}_B$: Bézier basis matrix

$\mathbf{P}$: Control point vector

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}_2 \\ \mathbf{P}_3 \end{bmatrix}$$

Why Use Matrix Form?

Computational Advantages

1. Efficiency:

- Single matrix multiplication
- Vectorized operations
- GPU-friendly

2. Derivatives:

- Easy to compute tangents
- First derivative:
$$\mathbf{B}'(t) = \mathbf{T}'(t) \cdot \mathbf{M}_B \cdot \mathbf{P}$$

- Where
$$\mathbf{T}'(t) = \begin{bmatrix} 0 & 1 & 2t & 3t^2 \end{bmatrix}$$

3. Transformations:

- Apply transforms to control points
- Curve transforms automatically

Implementation Example

```
import numpy as np

def bezier_cubic(t, control_points):
    # Parameter vector
    T = np.array([1, t, t**2, t**3])

    # Bézier basis matrix
    M = np.array([
        [ 1,  0,  0,  0],
        [-3,  3,  0,  0],
        [ 3, -6,  3,  0],
        [-1,  3, -3,  1]
    ])

    # Control points (4x2 for 2D)
    P = np.array(control_points)

    return T @ M @ P

# Usage
points = [[0,0], [1,2], [3,1], [4,0]]
curve_point = bezier_cubic(0.5, points)
```


Applications

Bézier Curves in Typography

Font Technologies

TrueType Fonts:

- Use quadratic Bézier curves
- Efficient rendering
- Compact file sizes

OpenType/PostScript:

- Use cubic Bézier curves
- Higher precision
- Better curve quality

Advantages:

- Scalable to any size
- Smooth curves at all resolutions
- Mathematical precision

[Font rendering example]



Letter 'S' with Bézier curves

Software Applications

Adobe Illustrator:

- Primary tool for vector graphics
- Pen tool creates Bézier paths
- Professional logo and illustration design

SVG (Scalable Vector Graphics):

- Web standard for vector graphics
- XML-based format using Bézier curves
- Scalable for responsive design

CAD Software:

- AutoCAD, SolidWorks, Rhino
- Precise engineering drawings
- NURBS surfaces based on Bézier

Animation Software:

- After Effects, Blender
- Smooth motion paths
- Character rigging and deformation

[Bézier curve in Adobe Illustrator]

[SVG logo example]



Vector logo design

Motion Design

↳ Easing Functions:

- CSS cubic-bezier() function
- Smooth transitions and animations
- Natural motion feel

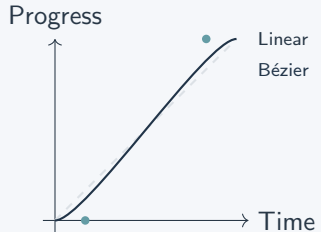
Motion Paths:

- Object trajectories in animation
- Camera movement in 3D
- Particle system paths

UI Animations:

- Button hover effects
- Page transitions
- Loading animations

Easing Curve Visualization



Smooth acceleration/deceleration

Motion Path Example

3D Applications

ı NURBS Surfaces:

- Non-Uniform Rational B-Splines
- Built on Bézier foundation
- Industry standard for CAD

Character Modeling:

- Smooth organic shapes
- Facial animation curves
- Rigging and deformation

Product Design:

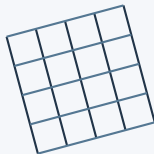
- Automotive body design
- Consumer electronics
- Architectural elements

Manufacturing:

- CNC machining paths

[Car body design with NURBS]

[3D character modeling]



NURBS surface patch

Properties & Limitations

Desirable Properties of Curve Systems

What Makes a Good Curve?

1. Intuitive Control:

- Control points have geometric meaning
- Predictable curve behavior
- Easy to achieve desired shapes

2. Local vs Global Control:

- **Local:** Changing one control point affects only nearby curve
- **Global:** Changes affect entire curve

3. Smoothness:

- Continuous derivatives
- No unwanted oscillations

Local vs Global Control

Global Control



Moving P_1 affects entire curve

Local Influence



Changes affect only nearby region

Bézier Curves: The Global Control Problem

Bézier Limitations

1. Global Influence:

- Every control point affects entire curve
- Cannot make local modifications
- Difficult to edit complex shapes

2. High Degree Problems:

- More control points = higher degree
- Potential for unwanted oscillations
- Numerical instability

3. Limited Flexibility:

- Fixed number of control points per segment
- Cannot represent some curves exactly (circles)
- Joining multiple segments requires care



High degree = problems

Enter B-Splines: Local Control

B-Spline Advantages

Local Control:

- Each control point affects only nearby curve segment
- Easy local modifications
- Predictable editing behavior

Flexible Degree:

- Can have many control points with low degree
- No unwanted oscillations
- Numerically stable

Continuity Control:

- Automatic smoothness between segments
- Can control continuity level
- Professional modeling standard



Many points, low degree, smooth

When to Use What?

Curve Selection Guide

Application	Bézier Curves	B-Splines	NURBS
Simple shapes	✓ Good	- Overkill	- Overkill
Typography	✓ Perfect	- Too complex	- Too complex
UI Animation	✓ Ideal	- Unnecessary	- Unnecessary
Complex curves	✗ Difficult	✓ Excellent	✓ Best
CAD/Engineering	✗ Limited	✓ Good	✓ Standard
3D Surfaces	✗ Patches only	✓ Good	✓ Perfect

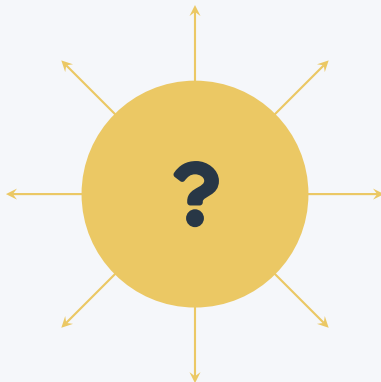
Recommendation

Start with Bézier: Simple, intuitive, widely supported

Move to B-splines: When you need complex shapes with local control

Use NURBS: For professional CAD and precise geometric modeling

Questions?



References & Further Reading



Peter Shirley and Steve Marschner et al. *Fundamentals of Computer Graphics (4th Edition)*. CRC Press, 2016.

Available as PDF



Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation (4th Edition)*. Morgan Kaufmann, 2023.

Available online



Peter Shirley. *Ray Tracing in One Weekend*. Self-published, 2016–2020.

Project Website



MIT OpenCourseWare: 6.837 Computer Graphics.
ocw.mit.edu/6-837



Scratchapixel: Learn Computer Graphics Programming.
scratchapixel.com